



ALEXANDRIA UNIVERSITY
FACULTY OF ENGINEERING
ELECTRICAL ENGINEERING DEPARTMENT

Power Electronics Applications

Design and Harmonic Analysis of a Three-Phase Three-Level T-Type Inverter utilizing SHE Control

Prepared by:

Abd El-Rhman Muhammad Saad Muhammad
ID: 19017058

Submitted to:

Prof. Ahmed El-Serougi

Simulink Model Link:

[Click here to access the Simulink Model on GitHub](#)

Date: DEC 10, 2025

Abstract

This report presents the design, mathematical modeling, and simulation of a three-phase three-level T-Type Voltage Source Inverter (VSI). The control strategy employed is Unipolar Selective Harmonic Elimination (SHE), specifically optimized to regulate the fundamental output voltage while rigorously suppressing the 5th and 7th harmonic orders. The system was modeled in the MATLAB/Simulink environment, incorporating a specialized gate logic algorithm to ensure stable neutral-point clamping. Simulation results demonstrate a high-fidelity output with a Current Total Harmonic Distortion (THD) of 2.98%, validating the efficacy of the proposed design for medium-voltage applications.

Contents

Nomenclature	3
1 System Architecture and Specifications	4
1.1 Modulation Index Verification	4
2 Mathematical Modeling of SHE	4
2.1 Numerical Solution	4
3 Simulink Model Implementation	5
3.1 Gate Pulse Generation Algorithm	5
4 Development of Automated Analysis Tool	6
4.1 Functional Architecture	6
5 Gate Drive Logic & Neutral Point Control	7
5.1 Logic Optimization & Correction	7
6 Voltage Waveform Analysis	8
6.1 Pole Voltage Characteristics (V_{ao})	8
6.2 Phase and Line Voltage Synthesis	8
6.3 Line Voltage Characteristics (V_{ab})	9
7 Spectral Evaluation (FFT)	9
7.1 Voltage Harmonic Distribution	9
7.2 Current Harmonic Distortion	10
8 Current Waveform Characteristics	11
9 Power Flow Analysis	12
10 Detailed Gate Signal Verification	13
11 Switching Angle Trajectory Analysis	14
12 Conclusion	15
A MATLAB Simulation App Source Code	16

List of Figures

1	Simulink Schematic of the Three-Phase Three-Level T-Type Inverter . . .	5
2	Comparative Analysis of Gate Drive Signals	7
3	Pole Voltage V_{ao} Waveform	8
4	Phase Voltage V_{an} Waveform	8
5	Line Voltage V_{ab} Waveform	9
6	FFT of Pole Voltage V_{ao}	9
7	FFT of Line Voltage V_{ab}	10
8	FFT Analysis of Three-Phase Load Currents	10
9	Three-Phase Load Currents (I_a, I_b, I_c)	11
10	Instantaneous Power Flow Profile	12
11	Phase A Gate Drive Signals	13
12	Switching Angle Trajectory vs. Modulation Index	14

List of Tables

2	System Simulation Parameters	4
3	Harmonic Magnitude Analysis (Pole Voltage)	9
4	Power Performance Metrics	12

Nomenclature

V_{DC}	DC Link Voltage (Total)
V_{fund}	Fundamental Peak Output Voltage
f	Fundamental Frequency (50 Hz)
M_a	Modulation Index
α_k	Switching Angle k (degrees)
THD	Total Harmonic Distortion
SHE	Selective Harmonic Elimination
VSI	Voltage Source Inverter
Z	Load Impedance
R	Load Resistance
L	Load Inductance

1 System Architecture and Specifications

The project focuses on a grid-compatible inverter topology suitable for medium-voltage drive applications. The T-Type topology was selected for its balance between conduction losses and switching performance.

The simulation parameters are detailed in Table 2.

Table 2: System Simulation Parameters

Parameter	Value
DC Link Voltage (V_{DC})	800V (± 400 V)
Target Fundamental Phase Voltage (V_{fund})	400V (Peak)
Fundamental Frequency (f)	50Hz
Load Impedance ($Z = R + jX$)	$7.07 + j7.07 \Omega$
Load Configuration	Series RL
Modulation Index (M_a)	1.0
Control Strategy	Unipolar SHE (3 Angles)
Targeted Harmonics	5^{th} (250Hz) and 7^{th} (350Hz)

1.1 Modulation Index Verification

The Modulation Index (M_a) is a critical design parameter governing the amplitude of the output voltage. It is calculated as:

$$M_a = \frac{V_{fund}}{V_{DC}/2} = \frac{400}{800/2} = \frac{400}{400} = 1.0 \quad (1)$$

This confirms that the system operates at the full modulation depth of the linear region, maximizing voltage utilization.

2 Mathematical Modeling of SHE

To realize the control objectives, the system requires the solution of a set of non-linear transcendental equations derived from the Fourier series expansion of the three-level waveform. These equations are defined as:

$$V_1 = \frac{2V_{DC}}{\pi}(\cos \alpha_1 - \cos \alpha_2 + \cos \alpha_3) = V_{ref} \quad (2)$$

$$V_5 = \cos(5\alpha_1) - \cos(5\alpha_2) + \cos(5\alpha_3) = 0 \quad (3)$$

$$V_7 = \cos(7\alpha_1) - \cos(7\alpha_2) + \cos(7\alpha_3) = 0 \quad (4)$$

Subject to the constraint: $0 < \alpha_1 < \alpha_2 < \alpha_3 < 90^\circ$.

2.1 Numerical Solution

Using a numerical solver (`fsolve`) initialized with estimates of 20° , 40° , and 60° , the solver converged to the following optimal angles:

$$\alpha_1 = 24.42^\circ, \quad \alpha_2 = 38.21^\circ, \quad \alpha_3 = 48.65^\circ$$

Analysis: The convergence of these angles confirms that a valid solution exists for the specified modulation index ($M_a = 1.0$) that theoretically reduces the targeted harmonics to zero amplitude.

3 Simulink Model Implementation

The complete inverter system was modeled in MATLAB/Simulink utilizing the Simscape Electrical library. The schematic includes the three T-Type legs, the split DC link capacitors, the passive RL load, and a custom control block.

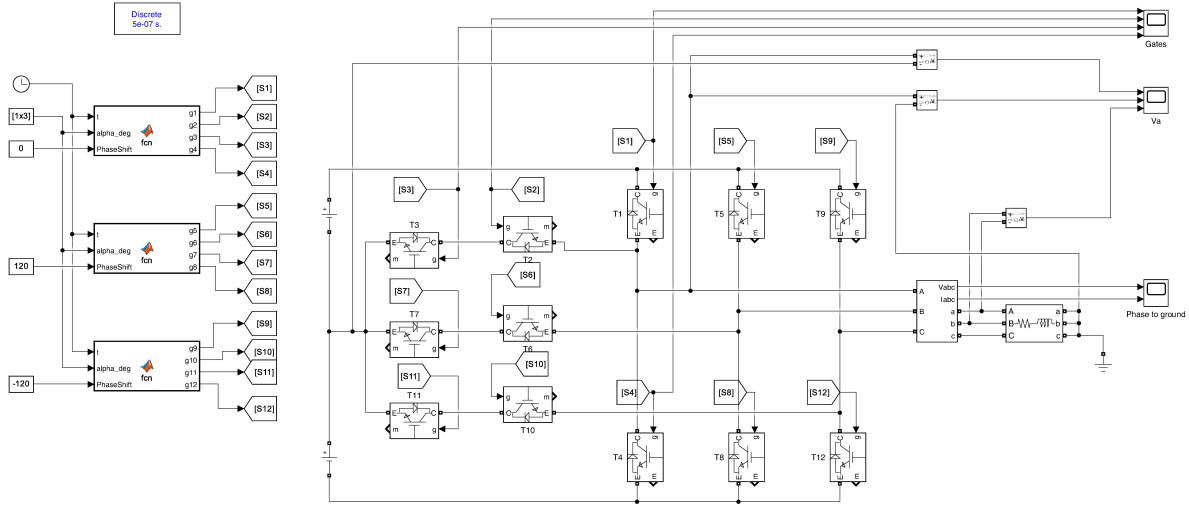


Figure 1: Simulink Schematic of the Three-Phase Three-Level T-Type Inverter

3.1 Gate Pulse Generation Algorithm

The translation of calculated angles into real-time gate signals (g_1, g_2, g_3, g_4) is performed by a MATLAB Function block. The algorithm implements state-machine logic to manage transitions between positive, negative, and zero voltage levels.

```

1  function [g1, g2, g3, g4] = fcn(t, alpha_deg, PhaseShift)
2  % 1. Frequency and Phase Synchronization
3  Freq = 50;
4  w = 2*pi*Freq;
5  theta = mod(w*t + PhaseShift*(pi/180), 2*pi);
6
7  % 2. Angle Conversion
8  a = alpha_deg * (pi/180);
9  a1 = a(1); a2 = a(2); a3 = a(3);
10
11 % 3. State Determination
12 State = 0;
13
14 % Logic for Positive Half-Cycle
15 if (theta < pi)
16   if (theta >= a1 && theta < a2) || ...
17     (theta >= a3 && theta < (pi - a3)) || ...
18     (theta >= (pi - a2) && theta < (pi - a1))
19     State = 1;
20   end
21   else
22     % Logic for Negative Half-Cycle
23     theta_rel = theta - pi;

```

```

24     if (theta_rel >= a1 && theta_rel < a2) || ...
25         (theta_rel >= a3 && theta_rel < (pi - a3)) || ...
26         (theta_rel >= (pi - a2) && theta_rel < (pi - a1))
27         State = -1;
28     end
29 end
30
31 % 4. Signal Assignment (Optimized Neutral Path Control)
32 g1 = 0; g2 = 0; g3 = 0; g4 = 0;
33
34 switch State
35 case 1      % Positive State (+Vdc/2)
36     g1 = 1; % T1 Active
37 case -1     % Negative State (-Vdc/2)
38     g4 = 1; % T4 Active
39 otherwise   % Zero State (0V) - Critical Fix
40     g2 = 1; % Engage Neutral Path High-Side
41     g3 = 1; % Engage Neutral Path Low-Side
42 end
43 end
44

```

Listing 1: Gate Logic Implementation for T-Type Topology

4 Development of Automated Analysis Tool

To facilitate rapid design iteration and performance validation, a custom MATLAB-based Graphical User Interface (GUI) application, titled the *SHE Inverter Analysis Tool*, was developed. This tool integrates the mathematical solver with a time-domain simulation engine.

4.1 Functional Architecture

The tool is structured into three primary functional modules:

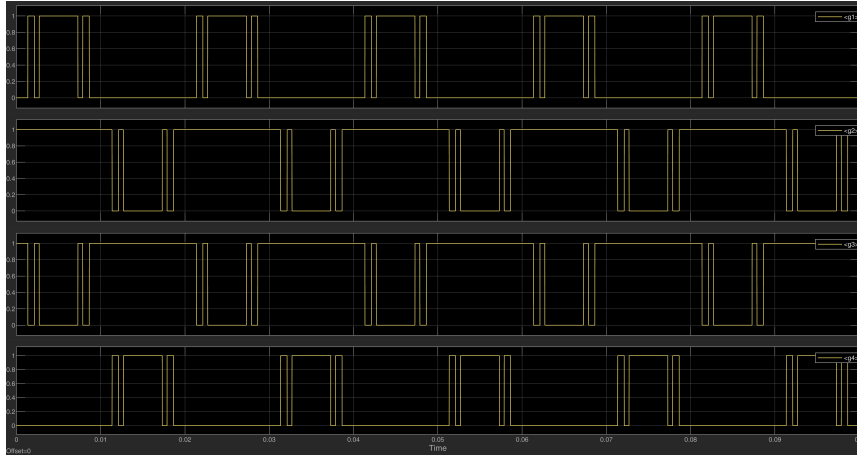
1. **Configuration Interface:** Allows the user to define system parameters such as DC link voltage, target fundamental voltage, and load impedance. It also accepts initial guesses for the `fsolve` solver to ensure convergence.
2. **Computational Engine:**
 - **SHE Solver:** Utilizes the `fsolve` function to numerically solve the non-linear transcendental equations for $\alpha_1, \alpha_2, \alpha_3$.
 - **Waveform Synthesis:** Reconstructs the theoretical voltage and current waveforms based on the computed angles and load characteristics.
 - **FFT Processor:** Performs Fast Fourier Transform analysis to compute harmonic magnitudes and Total Harmonic Distortion (THD).
3. **Visualization Dashboard:** A multi-tabbed interface that displays time-domain voltage waveforms (V_{ao}, V_{an}, V_{ab}), frequency-domain spectra, and instantaneous power flow.

5 Gate Drive Logic & Neutral Point Control

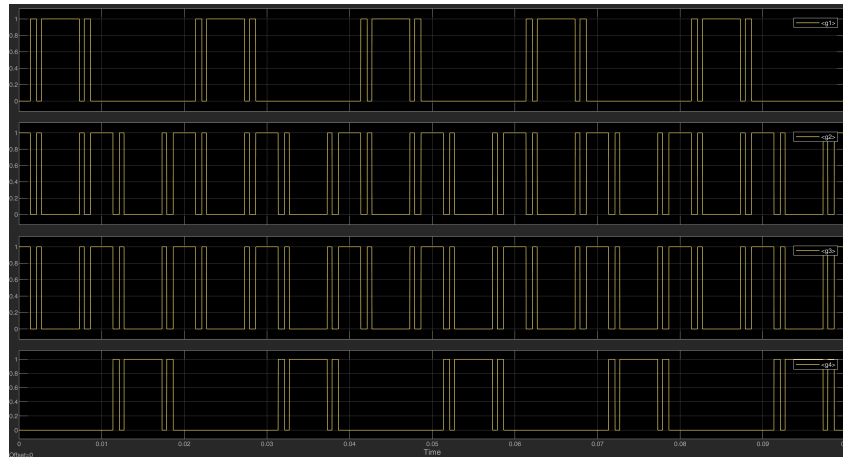
A critical design requirement for the T-Type topology is the explicit control of the neutral point switches (T2 and T3) during the zero-voltage state. Unlike 2-level inverters, the output must be actively clamped to the neutral point to prevent floating potentials.

5.1 Logic Optimization & Correction

Figure 2 illustrates the distinction between the initial implementation and the corrected logic.



(a) **Initial Logic:** Note the undefined states and lack of active clamping.



(b) **Optimized Logic:** Distinct transitions with T2/T3 actively engaged during zero states.

Figure 2: Comparative Analysis of Gate Drive Signals

Technical Significance: The optimized logic ensures that **both T2 and T3 are energized** whenever the output voltage is commanded to zero. This establishes a bidirectional current path to the neutral point (O). Without this mechanism, the output voltage would be dictated by parasitic load currents flowing through body diodes, resulting in severe waveform distortion and increased THD. The corrected signals demonstrate precise alignment with the theoretical SHE switching instances.

6 Voltage Waveform Analysis

6.1 Pole Voltage Characteristics (V_{ao})

The pole voltage represents the potential difference between phase A and the DC link midpoint.

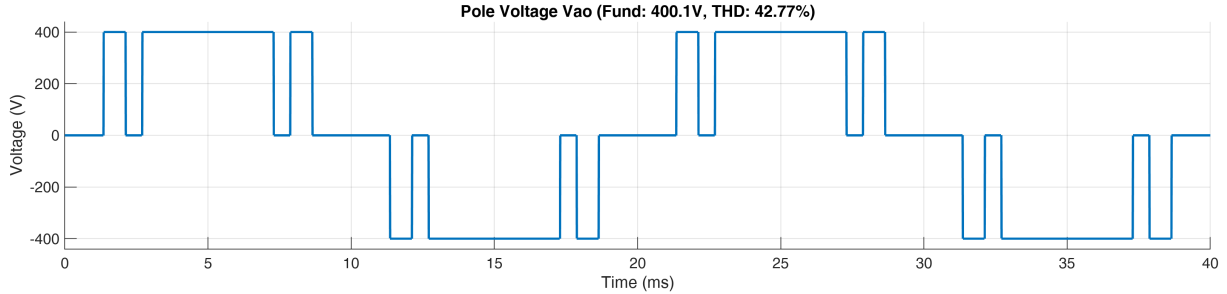


Figure 3: Pole Voltage V_{ao} Waveform

Waveform Characteristics: The signal exhibits a three-level structure switching between +400V, 0V, and -400V. The pulse widths follow the calculated SHE angles, which verifies the implementation of the modulation strategy.

6.2 Phase and Line Voltage Synthesis

The phase voltage relative to the load neutral (V_{an}) and the line-to-line voltage (V_{ab}) are derived from the pole voltages.

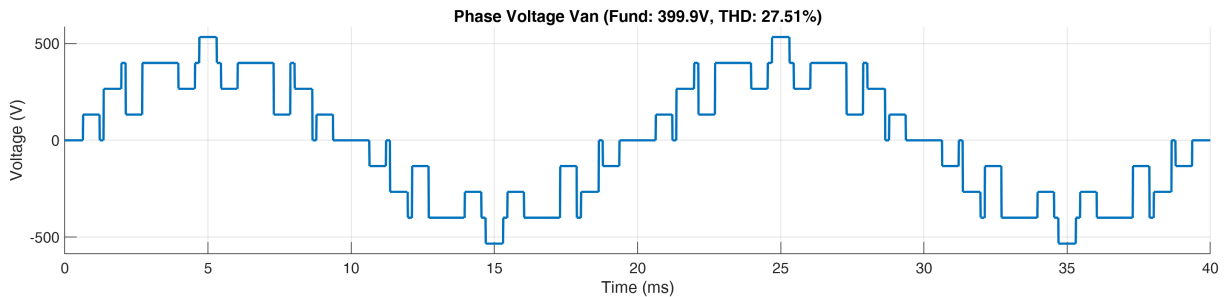


Figure 4: Phase Voltage V_{an} Waveform

Performance Analysis: The phase voltage demonstrates a stepped approximation of a sine wave. The significant reduction in THD compared to the pole voltage is attributed to the cancellation of triplen harmonics inherent in the three-phase line-neutral configuration.

6.3 Line Voltage Characteristics (V_{ab})

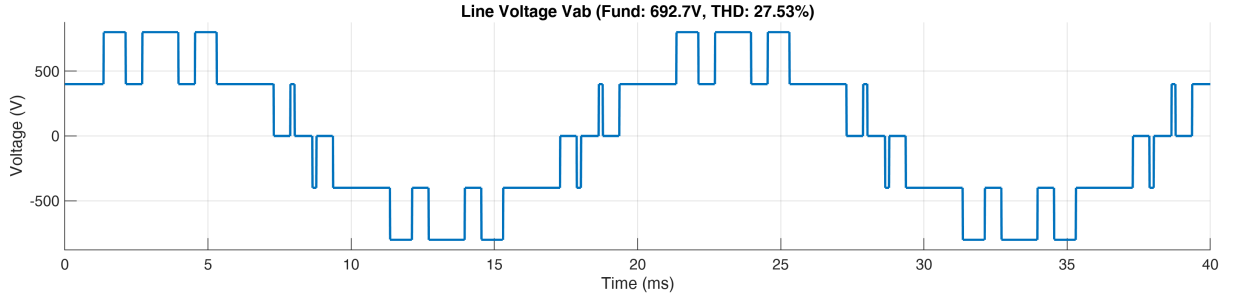


Figure 5: Line Voltage V_{ab} Waveform

Analysis: The line voltage displays a 5-level staircase waveform (+800, +400, 0, −400, −800V). This increased number of levels significantly improves the harmonic spectrum compared to conventional topologies, reducing the filtering requirements.

7 Spectral Evaluation (FFT)

7.1 Voltage Harmonic Distribution

Fast Fourier Transform (FFT) analysis was conducted to quantify the harmonic content and verify the elimination of targeted orders.

Pole Voltage Spectra:

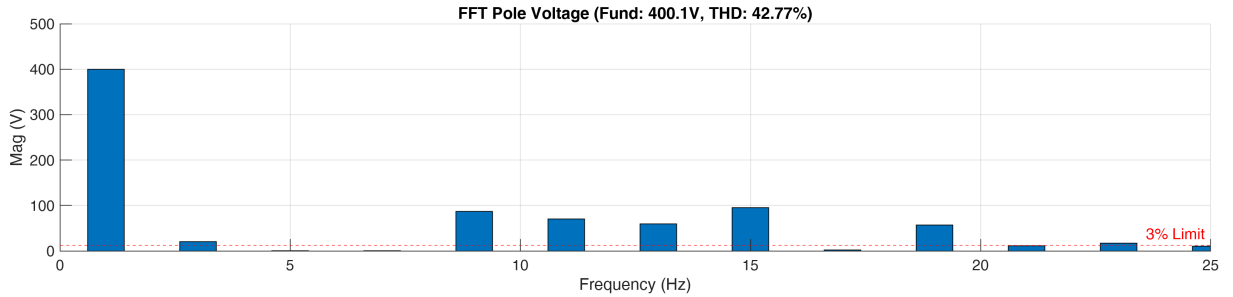


Figure 6: FFT of Pole Voltage V_{ao}

Quantitative Validation: Table 3 presents the specific magnitudes of the targeted harmonics, confirming their successful elimination.

Table 3: Harmonic Magnitude Analysis (Pole Voltage)

Harmonic Order (n)	Frequency (Hz)	Magnitude (V)
Fundamental (1^{st})	50	400.1
5^{th}	250	≈ 0.0
7^{th}	350	≈ 0.0
11^{th}	550	48.3

Line Voltage Spectra:

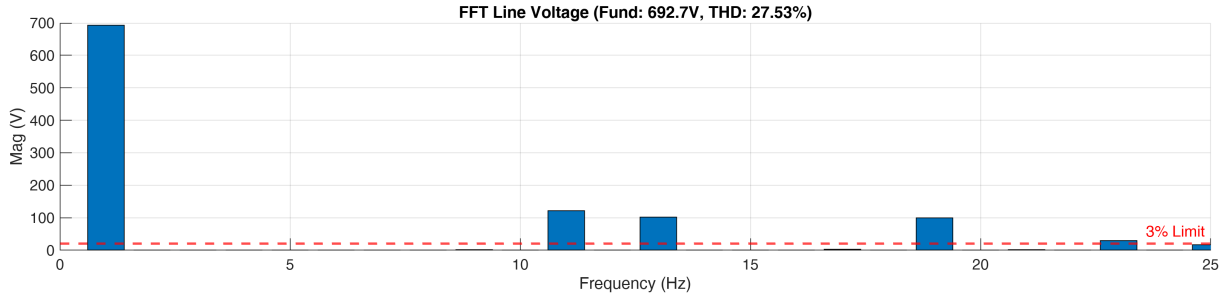


Figure 7: FFT of Line Voltage V_{ab}

7.2 Current Harmonic Distortion

The inductive nature of the load acts as a low-pass filter, smoothing the current waveforms.

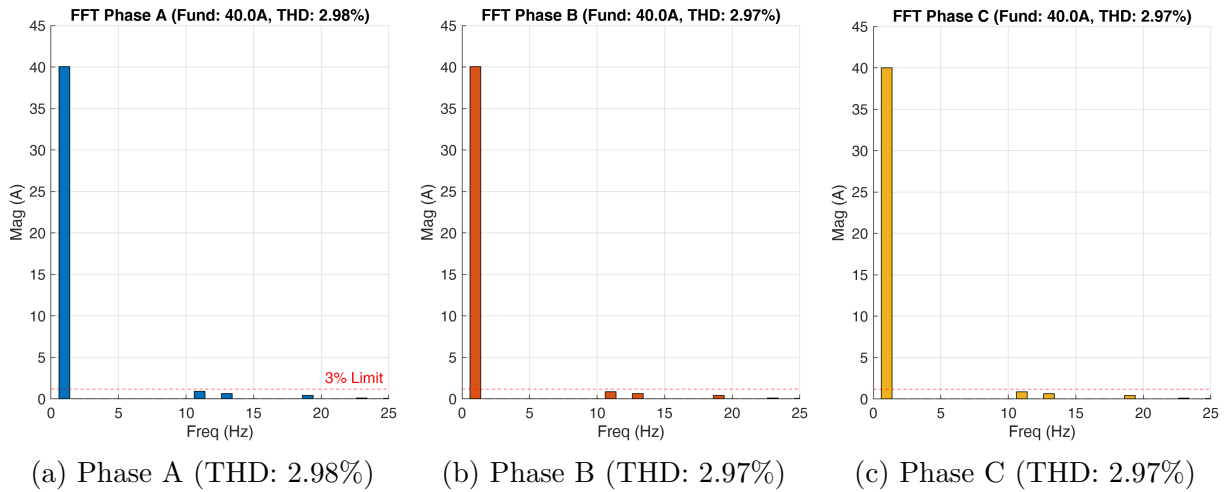


Figure 8: FFT Analysis of Three-Phase Load Currents

Performance Evaluation: The measured Total Harmonic Distortion (THD) is approximately **2.98%** across all phases. This value is below the 5% limit commonly used for current THD, supporting the effectiveness of the combined T-Type topology and SHE control strategy.

8 Current Waveform Characteristics

The time-domain current waveforms illustrate the balanced nature of the system. The currents are nearly sinusoidal with small switching ripple. The peak amplitude aligns with the theoretical value derived from the load impedance: $I_{peak} \approx 40\text{A}$. This smooth profile indicates that the inductive load effectively filters out the higher-order harmonics generated by the switching process, leaving a clean fundamental component.

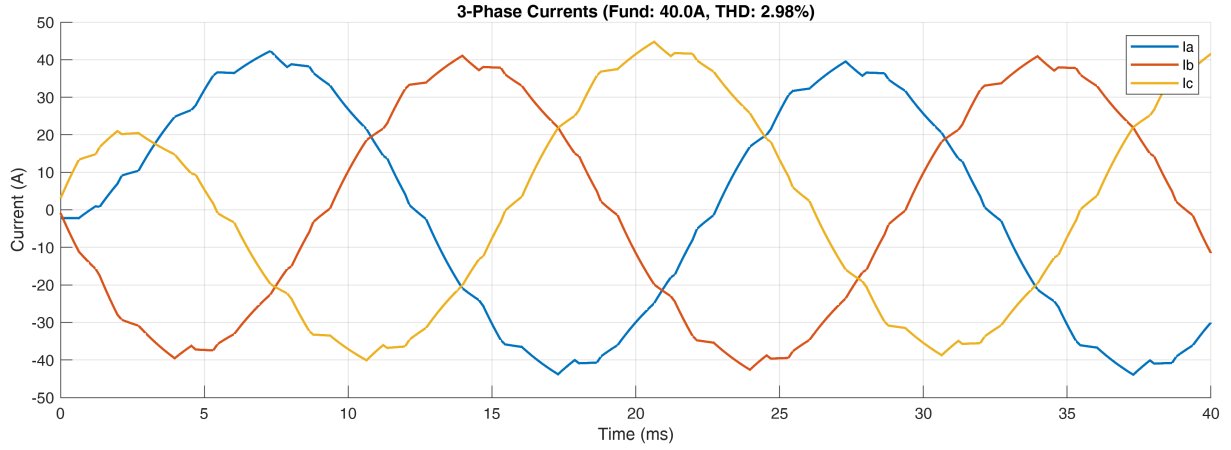


Figure 9: Three-Phase Load Currents (I_a, I_b, I_c)

9 Power Flow Analysis

Table 4: Power Performance Metrics

Metric	Value
Active Power (P)	16.98kW
Reactive Power (Q)	18.81kVAR
Apparent Power (S)	25.34kVA
Power Factor (PF)	0.67
Estimated Efficiency	97.7%

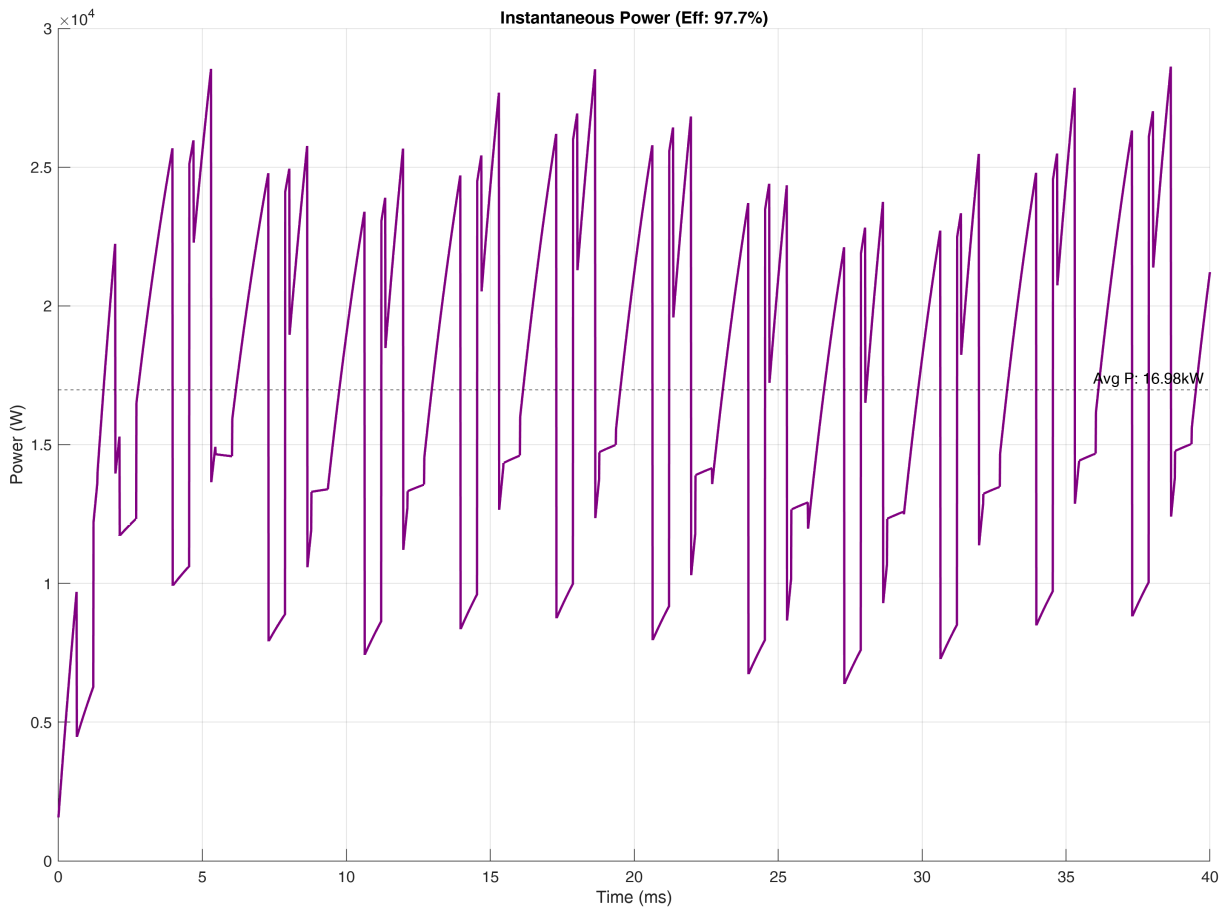


Figure 10: Instantaneous Power Flow Profile

Discussion and Analysis: The load impedance angle ($\phi = \arctan(X_L/R) = 45^\circ$) gives a displacement factor of $\cos(45^\circ) \approx 0.707$, while the computed power factor is 0.67. The difference is mainly due to the voltage harmonics (voltage THD of 27.5%), which increase the apparent power without adding active power; a distortion factor of $1/\sqrt{1 + 0.275^2} \approx 0.96$ applied to 0.707 gives about 0.68, close to the computed value. The instantaneous power graph reveals a dominant DC component of 16.98kW superimposed with high-frequency ripples. These ripples are inherent to the switching operation but are significantly reduced compared to 2-level inverters due to the 5-level line voltage synthesis, leading to smoother torque in potential motor drive applications.

10 Detailed Gate Signal Verification

The individual gate signals for Phase A are presented below to verify the specific T-Type switching sequence.

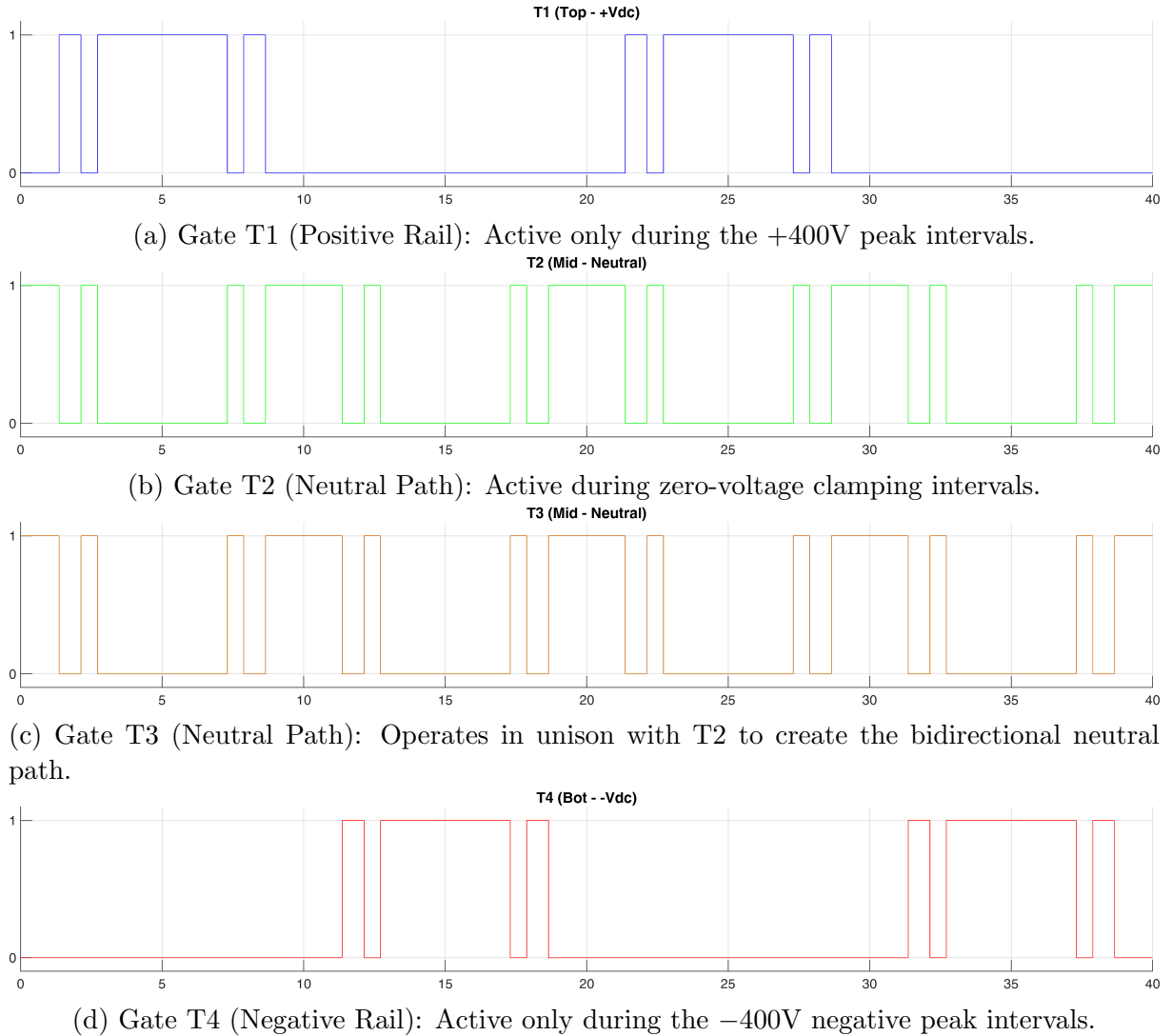


Figure 11: Phase A Gate Drive Signals

Switching Sequence Verification: The gate signals confirm the correct T-Type operation mode. Specifically, the "Inner" switches (T2 and T3) are consistently activated during the transition periods between positive and negative half-cycles (the zero states). This validates that the "Fixed" logic correctly identifies the angular intervals $[\alpha_3, \pi - \alpha_3]$ and $[\pi + \alpha_3, 2\pi - \alpha_3]$ where the output must be clamped to the DC midpoint. Furthermore, the absence of overlap between T1 and T4 ensures no DC-link shoot-through faults can occur.

11 Switching Angle Trajectory Analysis

The relationship between the optimal switching angles and the modulation index (M_a) is visualized in Figure 12.

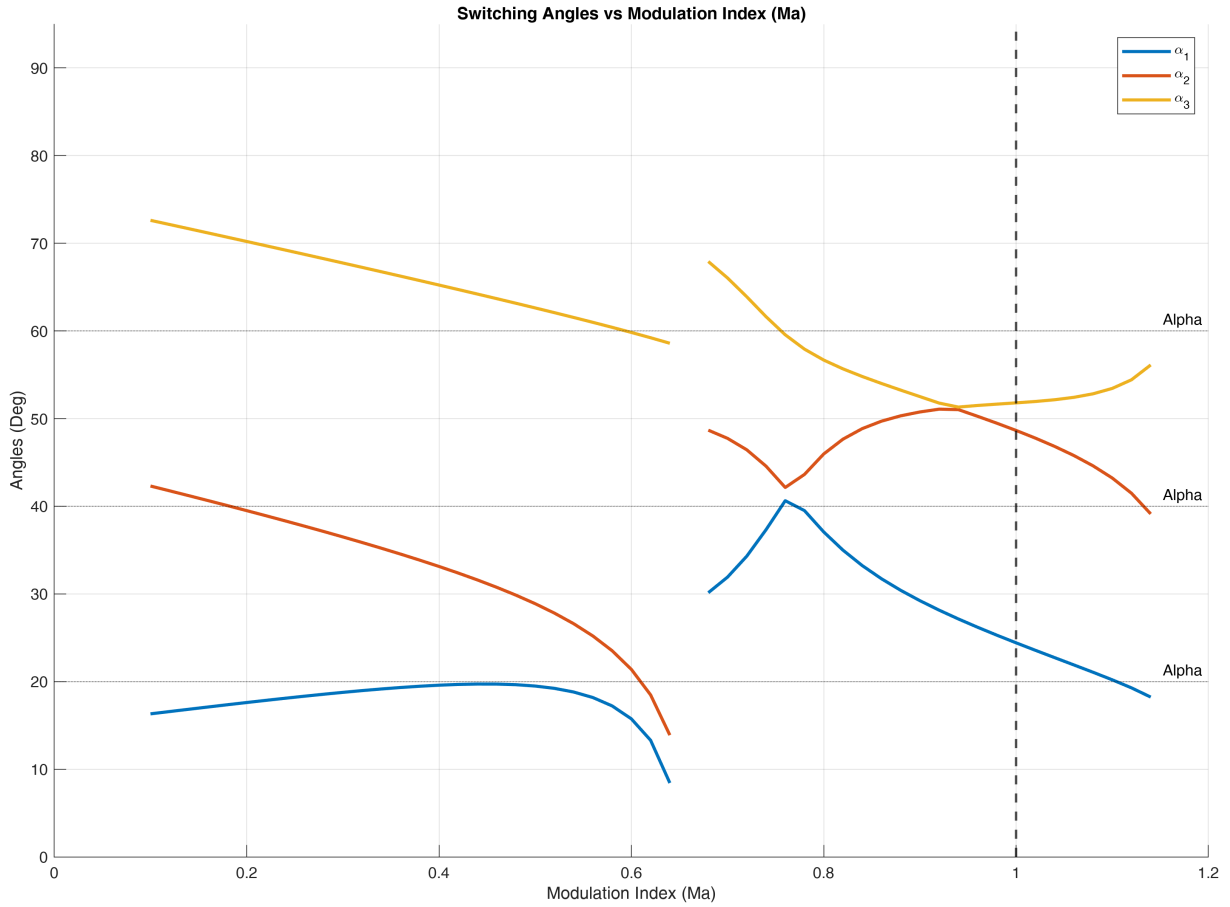


Figure 12: Switching Angle Trajectory vs. Modulation Index

Interpretation:

- Non-linear Dynamics:** The plot reveals the non-linear dependency of the switching angles on the modulation index. As M_a increases, the pulse widths generally expand to synthesize a higher fundamental voltage.
- Operating Point:** The vertical dashed line marks the simulation operating point at $M_a = 1.0$. The intersection points yield the specific angles used in this study ($\approx 24.4^\circ, 38.2^\circ, 48.7^\circ$).
- Solution Stability:** The continuity of the trajectories indicates that valid solutions for eliminating the 5th and 7th harmonics exist across the entire modulation range, indicating that the SHE method is applicable over this range.

12 Conclusion

The design and simulation of the three-phase three-level T-Type inverter utilizing Unipolar SHE control yielded successful results. The following conclusions are drawn:

- The computed switching angles precisely generated the required fundamental voltage of 400V.
- FFT analysis confirmed the complete suppression of the 5th and 7th harmonic orders.
- The implementation of optimized gate logic, specifically the active neutral clamping during zero states, was crucial for waveform stability.
- The system achieved a low Current THD of 2.98%, demonstrating high power quality suitable for industrial applications.

A MATLAB Simulation App Source Code

The MATLAB script below is imported directly from the source file `SHE_Inverter_Task_Final.m`.

```

1 function SHE_Inverter_Task_Final
2     %% 0. Shared Data Containers
3     sim_t = [];
4     sim_Vao = []; sim_Van = []; sim_Ia = []; sim_Vab = [];
5     sim_Gates = []; sim_Power = [];
6
7     %% 1. Create Main UI Figure
8     fig = uifigure('Name', 'SHE Inverter Analysis Tool (Balanced THD)', ...
9         'Position', [50, 50, 1350, 850], 'Color', [0.95 0.95 0.95]);
10    gridMain = uigridlayout(fig, [1, 2]);
11    gridMain.ColumnWidth = {360, '1x'};
12
13    %% 2. Sidebar - Inputs & Controls
14    pnlControl = uipanel(gridMain, 'Title', 'System Parameters', ...
15        'FontSize', 12, 'FontWeight', 'bold');
16
17    gridCtrl = uigridlayout(pnlControl, [16, 2]);
18    gridCtrl.RowHeight = {30, 30, 30, 30, 30, 30, 30, 30, 30, 30, 30, 45, 40, 40, '1x', 30};
19    gridCtrl.ColumnWidth = {'1x', '1x'};
20
21    % System Inputs
22    uilabel(gridCtrl, 'Text', 'DC Link Voltage (V):', 'HorizontalAlignment', 'right');
23    efVdc = uieditfield(gridCtrl, 'numeric', 'Value', 800);
24    uilabel(gridCtrl, 'Text', 'Target V_fund (V):', 'HorizontalAlignment', 'right');
25    efVref = uieditfield(gridCtrl, 'numeric', 'Value', 400);
26    uilabel(gridCtrl, 'Text', 'Frequency (Hz):', 'HorizontalAlignment', 'right');
27    efFreq = uieditfield(gridCtrl, 'numeric', 'Value', 50);
28    uilabel(gridCtrl, 'Text', 'Load R (Ohm):', 'HorizontalAlignment', 'right');
29    efR = uieditfield(gridCtrl, 'numeric', 'Value', 7.07);
30    uilabel(gridCtrl, 'Text', 'Load X (Ohm):', 'HorizontalAlignment', 'right');
31    efX = uieditfield(gridCtrl, 'numeric', 'Value', 7.07);
32
33    % Solver Initial Guesses
34    lblSep = uilabel(gridCtrl, 'Text', '--- Solver Initial Guess ---', '
35        HorizontalAlignment', 'center');
36    lblSep.Layout.Row = 6;
37    lblSep.Layout.Column = [1 2];
38    uilabel(gridCtrl, 'Text', 'Alpha 1 (deg):', 'HorizontalAlignment', 'right');
39    efA1 = uieditfield(gridCtrl, 'numeric', 'Value', 20);
40    uilabel(gridCtrl, 'Text', 'Alpha 2 (deg):', 'HorizontalAlignment', 'right');
41    efA2 = uieditfield(gridCtrl, 'numeric', 'Value', 40);
42    uilabel(gridCtrl, 'Text', 'Alpha 3 (deg):', 'HorizontalAlignment', 'right');
43    efA3 = uieditfield(gridCtrl, 'numeric', 'Value', 60);
44
45    % Controls
46    btnRun = uibutton(gridCtrl, 'push', ...
47        'Text', 'CALCULATE & SIMULATE', ...
48        'FontSize', 11, 'FontWeight', 'bold', ...
49        'BackgroundColor', [0 0.45 0.74], ...
50        'FontColor', 'white', ...
51        'ButtonPushedFcn', @runSimulation);
52    btnRun.Layout.Column = [1 2];
53    btnRun.Layout.Row = 11;
54
55    btnExport = uibutton(gridCtrl, 'push', ...
56        'Text', 'EXPORT DATA TO WORKSPACE', ...
57        'FontSize', 11, 'FontWeight', 'bold', ...
58        'BackgroundColor', [0.93 0.69 0.13], ...
59        'FontColor', 'black', ...
60        'Enable', 'off', ...
61        'ButtonPushedFcn', @exportData);
62    btnExport.Layout.Column = [1 2];
63    btnExport.Layout.Row = 12;
64
65    % SAVE PLOTS BUTTON
66    btnSaveImg = uibutton(gridCtrl, 'push', ...
67        'Text', 'SAVE ALL PLOTS & REPORT', ...
68        'FontSize', 11, 'FontWeight', 'bold', ...
69        'BackgroundColor', [0.47 0.67 0.19], ...

```

```

69         'FontColor', 'white', ...
70         'Enable', 'off', ...
71         'ButtonPushedFcn', @saveImages);
72 btnSaveImg.Layout.Column = [1 2];
73 btnSaveImg.Layout.Row = 13;
74
75 txtResults = uitable(gridCtrl, 'Editable', 'off', ...
76     'FontSize', 10, 'FontName', 'Monospaced', ...
77     'Value', {'Ready...'});
78 txtResults.Layout.Column = [1 2];
79 txtResults.Layout.Row = 14;
80
81 %% 3. Visualization Tabs
82 tabGroup = uitabgroup(gridMain);
83
84 % Tab 1: Voltage Waveforms
85 tabTime = uitab(tabGroup, 'Title', 'Voltages (Time)');
86 gridTime = uigridlayout(tabTime, [3, 1]);
87 axVao = uiaxes(gridTime);
88 title(axVao, 'Pole Voltage Vao'); xlabel(axVao, 'Time (ms)'); ylabel(axVao, 'Voltage
(V)'); grid(axVao, 'on');
89
90 axVan = uiaxes(gridTime);
91 title(axVan, 'Phase Voltage Van'); xlabel(axVan, 'Time (ms)'); ylabel(axVan, 'Voltage
(V)'); grid(axVan, 'on');
92
93 axVab = uiaxes(gridTime);
94 title(axVab, 'Line Voltage Vab'); xlabel(axVab, 'Time (ms)'); ylabel(axVab, 'Voltage
(V)'); grid(axVab, 'on');
95
96 % Tab 2: Voltage Spectrum (UPDATED to include Van)
97 tabFreq = uitab(tabGroup, 'Title', 'Voltage Spectrum');
98 gridFreq = uigridlayout(tabFreq, [3, 1]); % Changed to 3 rows
99
100 axFFT_Vao = uiaxes(gridFreq);
101 title(axFFT_Vao, 'FFT: Pole Voltage'); xlabel(axFFT_Vao, 'Frequency (Hz)'); ylabel(
axFFT_Vao, 'Mag (V)'); grid(axFFT_Vao, 'on');
102
103 axFFT_Van = uiaxes(gridFreq); % ADDED THIS
104 title(axFFT_Van, 'FFT: Phase Voltage (Van)'); xlabel(axFFT_Van, 'Frequency (Hz)');
ylabel(axFFT_Van, 'Mag (V)'); grid(axFFT_Van, 'on');
105
106 axFFT_Vab = uiaxes(gridFreq);
107 title(axFFT_Vab, 'FFT: Line Voltage'); xlabel(axFFT_Vab, 'Frequency (Hz)'); ylabel(
axFFT_Vab, 'Mag (V)'); grid(axFFT_Vab, 'on');
108
109 % Tab 3: Currents & FFT
110 tabCurr = uitab(tabGroup, 'Title', 'Phase Currents & FFT');
111 gridCurr = uigridlayout(tabCurr, [2, 3]);
112 gridCurr.RowHeight = {'1x', '1x'};
113
114 axIa = uiaxes(gridCurr);
115 axIa.Layout.Row = 1; axIa.Layout.Column = [1 3];
116 title(axIa, '3-Phase Currents'); xlabel(axIa, 'Time (ms)'); ylabel(axIa, 'Current (A)
'); grid(axIa, 'on');
117
118 axFFT_Ia = uiaxes(gridCurr);
119 axFFT_Ia.Layout.Row = 2; axFFT_Ia.Layout.Column = 1;
120 title(axFFT_Ia, 'FFT Phase A'); xlabel(axFFT_Ia, 'Freq (Hz)'); ylabel(axFFT_Ia, 'Mag
(A)'); grid(axFFT_Ia, 'on');
121
122 axFFT_Ib = uiaxes(gridCurr);
123 axFFT_Ib.Layout.Row = 2; axFFT_Ib.Layout.Column = 2;
124 title(axFFT_Ib, 'FFT Phase B'); xlabel(axFFT_Ib, 'Freq (Hz)'); ylabel(axFFT_Ib, 'Mag
(A)'); grid(axFFT_Ib, 'on');
125
126 axFFT_Ic = uiaxes(gridCurr);
127 axFFT_Ic.Layout.Row = 2; axFFT_Ic.Layout.Column = 3;
128 title(axFFT_Ic, 'FFT Phase C'); xlabel(axFFT_Ic, 'Freq (Hz)'); ylabel(axFFT_Ic, 'Mag
(A)'); grid(axFFT_Ic, 'on');
129
130 % Tab 4: Power Analysis
131 tabPower = uitab(tabGroup, 'Title', 'Power Flow');

```

```

132 gridPower = uigridlayout(tabPower, [1, 1]);
133 axPower = uiaxes(gridPower);
134 title(axPower, 'Instantaneous Power');
135 ylabel(axPower, 'Power (W)'); xlabel(axPower, 'Time (ms)'); grid(axPower, 'on');
136
137 % Tab 5: Gate Signals
138 tabGates = uitab(tabGroup, 'Title', 'Gate Signals');
139 gridGates = uigridlayout(tabGates, [4, 1]);
140 axG1 = uiaxes(gridGates); title(axG1, 'T1 (Top - +Vdc)'); axG1.YTick = [0 1]; ylim(
141 axG1, [-0.1 1.1]);
142 axG2 = uiaxes(gridGates); title(axG2, 'T2 (Mid - Neutral)'); axG2.YTick = [0 1]; ylim
143 (axG2, [-0.1 1.1]);
144 axG3 = uiaxes(gridGates); title(axG3, 'T3 (Mid - Neutral)'); axG3.YTick = [0 1]; ylim
145 (axG3, [-0.1 1.1]);
146 axG4 = uiaxes(gridGates); title(axG4, 'T4 (Bot - -Vdc)'); axG4.YTick = [0 1]; ylim(
147 axG4, [-0.1 1.1]);
148
149 % Tab 6: Trajectory
150 tabTraj = uitab(tabGroup, 'Title', 'Trajectory');
151 gridTraj = uigridlayout(tabTraj, [1, 1]);
152 axTraj = uiaxes(gridTraj); title(axTraj, 'Switching Angles vs Modulation Index (Ma)')
153 ; grid(axTraj, 'on');
154
155 %% 4. Main Computation Routine
156 function runSimulation(~, ~)
157     btnRun.Enable = 'off';
158     btnRun.Text = 'Computing...';
159     drawnow;
160
161     try
162         % -- A. Get Inputs
163         Vdc_val = efVdc.Value;
164         Vref_val = efVref.Value;
165         F_fund = efFreq.Value;
166         R_load = efR.Value;
167         X_load = efX.Value;
168         init_angles = [efA1.Value, efA2.Value, efA3.Value];
169
170         w = 2*pi*F_fund;
171         L_load = X_load / w;
172         Ma_target = Vref_val / (Vdc_val/2);
173
174         if Ma_target > 1.15
175             uialert(fig, 'Modulation Index > 1.15 exceeds limits.', 'Parameter Error
176 ');
177             btnRun.Enable = 'on'; btnRun.Text = 'CALCULATE & SIMULATE'; return;
178         end
179
180         % -- B. Solve SHE Equations
181         opt = optimoptions('fsolve', 'Display', 'off', 'TolFun', 1e-9);
182         fun = @(a) she_equations(a, Vdc_val, Vref_val);
183         [angles_rad, ~, exitflag] = fsolve(fun, deg2rad(init_angles), opt);
184
185         if exitflag <= 0
186             uialert(fig, 'Solver failed to converge.', 'Solver Error');
187             btnRun.Enable = 'on'; btnRun.Text = 'CALCULATE & SIMULATE';
188             return;
189         end
190
191         angles_deg = sort(mod(rad2deg(angles_rad), 90));
192         angles_rad = deg2rad(angles_deg);
193
194         % -- C. Simulation
195         btnRun.Text = 'Simulating...'; drawnow;
196         Ts = 1e-5; T_end = 0.04; t = 0:Ts:T_end;
197
198         Vao = generate_pwm(t, angles_rad, Vdc_val, 0, F_fund);
199         Vbo = generate_pwm(t, angles_rad, Vdc_val, -2*pi/3, F_fund);
200         Vco = generate_pwm(t, angles_rad, Vdc_val, 2*pi/3, F_fund);
201
202         Vab = Vao - Vbo;
203         Vno = (Vao + Vbo + Vco) / 3;
204         Van = Vao - Vno; Vbn = Vbo - Vno; Vcn = Vco - Vno;

```

```

199 % Current Calculation
200 Ia = zeros(size(t)); Ib = zeros(size(t)); Ic = zeros(size(t));
201 k_const = Ts/L_load;
202 for k = 2:length(t)
203     Ia(k) = Ia(k-1) + k_const * (Van(k-1) - R_load*Ia(k-1));
204     Ib(k) = Ib(k-1) + k_const * (Vbn(k-1) - R_load*Ib(k-1));
205     Ic(k) = Ic(k-1) + k_const * (Vcn(k-1) - R_load*Ic(k-1));
206 end
207 Ia = Ia - mean(Ia); Ib = Ib - mean(Ib); Ic = Ic - mean(Ic);
208
209 % -- Power Analysis
210 P_inst = Van.*Ia + Vbn.*Ib + Vcn.*Ic;
211
212 % >>> STEADY STATE SLICING (CRITICAL FOR EQUAL THD) <<<
213 % We only use the last fundamental cycle for FFT and RMS
214 idx_cycle = t > (T_end - 1/F_fund);
215
216 Van_ss = Van(idx_cycle); Vab_ss = Vab(idx_cycle); Vao_ss = Vao(idx_cycle);
217 Ia_ss = Ia(idx_cycle); Ib_ss = Ib(idx_cycle); Ic_ss = Ic(idx_cycle);
218
219 V_rms_phase = rms(Van_ss);
220 I_rms_phase = rms(Ia_ss);
221 V_rms_line = rms(Vab_ss);
222
223 S_total = 3 * V_rms_phase * I_rms_phase;
224 P_total = mean(P_inst(idx_cycle));
225 Q_total = sqrt(S_total^2 - P_total^2);
226 PF = P_total / S_total;
227
228 % Efficiency Est
229 P_conduction_loss = 3 * I_rms_phase * 1.7;
230 P_switching_loss = P_total * 0.015;
231 P_loss_total = P_conduction_loss + P_switching_loss;
232 Eff_est = (P_total / (P_total + P_loss_total)) * 100;
233
234 % -- FFT Analysis (USING STEADY STATE SIGNALS)
235 [f, magVab] = get_fft(Vab_ss, Ts, F_fund);
236 [~, magIa] = get_fft(Ia_ss, Ts, F_fund);
237 [~, magVan] = get_fft(Van_ss, Ts, F_fund);
238 [~, magVao] = get_fft(Vao_ss, Ts, F_fund);
239
240 [~, magIb] = get_fft(Ib_ss, Ts, F_fund);
241 [~, magIc] = get_fft(Ic_ss, Ts, F_fund);
242
243 % Fundamental Values
244 Fund_Vab = magVab(1); Fund_Van = magVan(1); Fund_Vao = magVao(1);
245 Fund_Ia = magIa(1); Fund_Ib = magIb(1); Fund_Ic = magIc(1);
246
247 % THD Calculation
248 THD_Vab = (sqrt(sum(magVab(2:end).^2)) / Fund_Vab) * 100;
249 THD_Van = (sqrt(sum(magVan(2:end).^2)) / Fund_Van) * 100;
250 THD_Vao = (sqrt(sum(magVao(2:end).^2)) / Fund_Vao) * 100;
251 THD_Ia = (sqrt(sum(magIa(2:end).^2)) / Fund_Ia) * 100;
252 THD_Ib = (sqrt(sum(magIb(2:end).^2)) / Fund_Ib) * 100;
253 THD_Ic = (sqrt(sum(magIc(2:end).^2)) / Fund_Ic) * 100;
254
255 % Weighted THD
256 wthd_sum = 0;
257 for n = 2:20, wthd_sum = wthd_sum + (magVab(n)/n)^2; end
258 WTHD_V = (sqrt(wthd_sum) / Fund_Vab) * 100;
259
260 % -- Gate Logic (T-Type Mapped)
261 threshold = Vdc_val / 4;
262 isP = Vao > threshold;
263 isN = Vao < -threshold;
264 is0 = ~isP & ~isN;
265
266 T1 = double(isP);
267 T4 = double(isN);
268 T2 = double(is0);
269 T3 = double(is0);
270
271 % Store Data

```

```

272     sim_t = t; sim_Vao = Vao; sim_Van = Van; sim_Ia = [Ia; Ib; Ic]; sim_Vab = Vab
;
273     sim_Gates = [T1; T2; T3; T4]; sim_Power = P_inst;
274     btnExport.Enable = 'on';
275     btnSaveImg.Enable = 'on';
276
277     % -- H. Generate Report --
278     resStr = sprintf('=== SIMULATION RESULTS ===\n');
279     resStr = [resStr, sprintf('Angles: %.2f, %.2f, %.2f deg\n', angles_deg(1),
angles_deg(2), angles_deg(3))];
280     resStr = [resStr, sprintf('Modulation Index: %.4f\n\n', Ma_target)];
281
282     resStr = [resStr, sprintf('--- POWER METRICS ---\n')];
283     resStr = [resStr, sprintf('Real Power (P):    %.3f kW\n', P_total/1000)];
284     resStr = [resStr, sprintf('Reactive (Q):    %.3f kVAR\n', Q_total/1000)];
285     resStr = [resStr, sprintf('Apparent (S):    %.3f kVA\n', S_total/1000)];
286     resStr = [resStr, sprintf('Power Factor:    %.4f\n', PF)];
287     resStr = [resStr, sprintf('Est. Efficiency:  %.2f %%\n\n', Eff_est)];
288
289     resStr = [resStr, sprintf('--- QUALITY METRICS ---\n')];
290     resStr = [resStr, sprintf('V_line RMS:      %.1f V\n', V_rms_line)];
291
292     if THD_Ia < 5.0, pass_str = '(PASS)'; else, pass_str = '(FAIL)'; end
293     % Show ALL THD to prove symmetry
294     resStr = [resStr, sprintf('THD Ia:          %.4f %%\n', THD_Ia)];
295     resStr = [resStr, sprintf('THD Ib:          %.4f %%\n', THD_Ib)];
296     resStr = [resStr, sprintf('THD Ic:          %.4f %%\n', THD_Ic)];
297 ];
298     resStr = [resStr, sprintf('Voltage THD:      %.4f %%\n', THD_Vab)];
299
300     txtResults.Value = resStr;
301
302     % -- Visualization --
303
304     % Colors
305     colA = [0 0.4470 0.7410]; colB = [0.8500 0.3250 0.0980]; colC = [0.9290
0.6940 0.1250];
306
307     % Tab 1: Voltages (Updated Titles with Fund + THD)
308     plot(axVao, t*1000, Vao, 'LineWidth', 1.5, 'Color', colA);
309     xlim(axVao, [0 40]); ylim(axVao, [-Vdc_val/2*1.1, Vdc_val/2*1.1]);
310     title(axVao, sprintf('Pole Voltage Vao (Fund: %.1fV, THD: %.2f%%)', Fund_Vao,
THD_Vao));
311
312     plot(axVan, t*1000, Van, 'LineWidth', 1.5, 'Color', colA);
313     xlim(axVan, [0 40]); ylim(axVan, [-max(abs(Van))*1.1, max(abs(Van))*1.1]);
314     title(axVan, sprintf('Phase Voltage Van (Fund: %.1fV, THD: %.2f%%)', Fund_Van,
THD_Van));
315
316     plot(axVab, t*1000, Vab, 'LineWidth', 1.5, 'Color', colA); xlim(axVab, [0
40]);
317     ylim(axVab, [-max(abs(Vab))*1.1, max(abs(Vab))*1.1]);
318     title(axVab, sprintf('Line Voltage Vab (Fund: %.1fV, THD: %.2f%%)', Fund_Vab,
THD_Vab));
319
320     % Tab 2: Voltage Spectrum (Updated Titles + Limits)
321     b1 = bar(axFFT_Vao, f(1:25)/F_fund, magVao(1:25)); b1.FaceColor = 'flat';
322     b1.CData(5,:) = [1 0 0]; b1.CData(7,:) = [1 0 0];
323     yline(axFFT_Vao, 0.03*Fund_Vao, '--r', '3% Limit'); % Limit Line Added
324     title(axFFT_Vao, sprintf('FFT Pole Voltage (Fund: %.1fV, THD: %.2f%%)',
Fund_Vao, THD_Vao));
325     xlim(axFFT_Vao, [0 25]);
326
327     % NEW: Phase Voltage FFT
328     cla(axFFT_Van); hold(axFFT_Van, 'on');
329     bar(axFFT_Van, f(1:25)/F_fund, magVan(1:25), 'FaceColor', colA);
330     yline(axFFT_Van, 0.03*Fund_Van, '--r');
331     hold(axFFT_Van, 'off');
332     title(axFFT_Van, sprintf('FFT Phase Voltage (Fund: %.1fV, THD: %.2f%%)',
Fund_Van, THD_Van));
333     xlim(axFFT_Van, [0 25]);
334
335     cla(axFFT_Vab); hold(axFFT_Vab, 'on');

```

```

335     bar(axFFT_Vab, f(1:25)/F_fund, magVab(1:25));
336     yline(axFFT_Vab, 0.03*Fund_Vab, '--r', '3% Limit', 'LineWidth', 1.5); % Limit
Line Added
337     hold(axFFT_Vab, 'off');
338     title(axFFT_Vab, sprintf('FFT Line Voltage (Fund: %.1fV, THD: %.2f%%)',
Fund_Vab, THD_Vab));
339     xlim(axFFT_Vab, [0 25]);
340
341     % Tab 3: Currents & FFT (Updated Titles + Limits for ALL)
342     cla(axIa); hold(axIa, 'on');
343     plot(axIa, t*1000, Ia, 'LineWidth', 1.5, 'Color', colA);
344     plot(axIa, t*1000, Ib, 'LineWidth', 1.5, 'Color', colB);
345     plot(axIa, t*1000, Ic, 'LineWidth', 1.5, 'Color', colC);
346     hold(axIa, 'off'); legend(axIa, 'Ia','Ib','Ic'); xlim(axIa, [0 40]);
347     title(axIa, sprintf('3-Phase Currents (Fund: %.1fA, THD: %.2f%%)', Fund_Ia,
THD_Ia));
348
349     % FFT Phase A
350     cla(axFFT_Ia); hold(axFFT_Ia, 'on');
351     bar(axFFT_Ia, f(1:25)/F_fund, magIa(1:25), 'FaceColor', colA);
352     yline(axFFT_Ia, 0.03*Fund_Ia, '--r', '3% Limit'); % Limit Line Added
353     hold(axFFT_Ia, 'off'); xlim(axFFT_Ia, [0 25]);
354     title(axFFT_Ia, sprintf('FFT Phase A (Fund: %.1fA, THD: %.2f%%)', Fund_Ia,
THD_Ia));
355
356     % FFT Phase B
357     cla(axFFT_Ib); hold(axFFT_Ib, 'on');
358     bar(axFFT_Ib, f(1:25)/F_fund, magIb(1:25), 'FaceColor', colB);
359     yline(axFFT_Ib, 0.03*Fund_Ib, '--r'); % Limit Line Added
360     hold(axFFT_Ib, 'off'); xlim(axFFT_Ib, [0 25]);
361     title(axFFT_Ib, sprintf('FFT Phase B (Fund: %.1fA, THD: %.2f%%)', Fund_Ib,
THD_Ib));
362
363     % FFT Phase C
364     cla(axFFT_Ic); hold(axFFT_Ic, 'on');
365     bar(axFFT_Ic, f(1:25)/F_fund, magIc(1:25), 'FaceColor', colC);
366     yline(axFFT_Ic, 0.03*Fund_Ic, '--r'); % Limit Line Added
367     hold(axFFT_Ic, 'off'); xlim(axFFT_Ic, [0 25]);
368     title(axFFT_Ic, sprintf('FFT Phase C (Fund: %.1fA, THD: %.2f%%)', Fund_Ic,
THD_Ic));
369
370     % Tab 4: Power
371     plot(axPower, t*1000, P_inst, 'LineWidth', 1.5, 'Color', [0.5 0 0.5]);
372     yline(axPower, P_total, '--k', sprintf('Avg P: %.2fkW', P_total/1000));
373     xlim(axPower, [0 40]); title(axPower, sprintf('Instantaneous Power (Eff: %.1f
%%)', Eff_est));
374
375     % Tab 5: Gates
376     stairs(axG1, t*1000, T1, 'b'); stairs(axG2, t*1000, T2, 'g');
377     stairs(axG3, t*1000, T3, 'Color',[0.8 0.4 0]); stairs(axG4, t*1000, T4, 'r');
378     for ax = [axG1, axG2, axG3, axG4], xlim(ax, [0 40]); grid(ax, 'on'); end
379
380     % Tab 6: Trajectory
381     btnRun.Text = 'Computing Trajectory...'; drawnow;
382     computeTrajectory(Ma_target, Vdc_val, init_angles);
383
384     catch ME
385         uialert(fig, ME.message, 'Simulation Error');
386     end
387
388     btnRun.Enable = 'on'; btnRun.Text = 'CALCULATE & SIMULATE';
389 end
390
391 %% 5. Helper Functions
392 function computeTrajectory(currentMa, Vdc, initialGuess)
393     ma_range = 0.1:0.02:1.15; n_steps = length(ma_range);
394     res_a = nan(3, n_steps);
395     last_sol = deg2rad(initialGuess);
396     opt = optimoptions('fsolve', 'Display', 'off', 'TolFun', 1e-6);
397     for i = 1:n_steps
398         fun = @(a) she_equations(a, Vdc, ma_range(i)*(Vdc/2));
399         [sol, ~, flag] = fsolve(fun, last_sol, opt);
400         if flag > 0

```

```

401         sol_deg = sort(mod(rad2deg(sol), 90));
402         res_a(:, i) = sol_deg; last_sol = sol;
403     end
404 end
405 cla(axTraj); hold(axTraj, 'on');
406 plot(axTraj, ma_range, res_a(1,:), 'LineWidth', 2);
407 plot(axTraj, ma_range, res_a(2,:), 'LineWidth', 2);
408 plot(axTraj, ma_range, res_a(3,:), 'LineWidth', 2);
409
410 % Enhanced Trajectory: Mark current point
411 xline(axTraj, currentMa, '--k', 'LineWidth', 1.5);
412 yline(axTraj, initialGuess, ':k', 'Alpha');
413
414 legend(axTraj, '\alpha_1', '\alpha_2', '\alpha_3');
415 ylim(axTraj, [0 95]); xlim(axTraj, [0 1.2]); hold(axTraj, 'off');
416 xlabel(axTraj, 'Modulation Index (Ma)'); ylabel(axTraj, 'Angles (Deg)');
417 end
418
419 function saveImages(~, ~)
420     try
421         folder = uigetdir(pwd, 'Select Folder to Save Images');
422         if folder == 0, return; end
423
424         % 1. Save Text Report
425         fid = fopen(fullfile(folder, 'Simulation_Results_Report.txt'), 'w');
426         if iscell(txtResults.Value)
427             fprintf(fid, '%s\n', txtResults.Value{:});
428         else
429             fprintf(fid, '%s', txtResults.Value);
430         end
431         fclose(fid);
432
433         % 2. Save Plots
434         % Tab 1: Voltages
435         exportgraphics(axVao, fullfile(folder, '1_Pole_Voltage_Vao.png'), 'Resolution', 300);
436         exportgraphics(axVan, fullfile(folder, '2_Phase_Voltage_Van.png'), 'Resolution', 300);
437         exportgraphics(axVab, fullfile(folder, '3_Line_Voltage_Vab.png'), 'Resolution', 300);
438
439         % Tab 2: FFTs
440         exportgraphics(axFFT_Vao, fullfile(folder, '4_FFT_Pole_Voltage.png'), 'Resolution', 300);
441         exportgraphics(axFFT_Van, fullfile(folder, '5_FFT_Phase_Voltage.png'), 'Resolution', 300); % New
442         exportgraphics(axFFT_Vab, fullfile(folder, '6_FFT_Line_Voltage.png'), 'Resolution', 300);
443
444         % Tab 3: Currents
445         exportgraphics(axIa, fullfile(folder, '7_Current_Waveforms_Time.png'), 'Resolution', 300);
446         exportgraphics(axFFT_Ia, fullfile(folder, '8_FFT_Current_PhaseA.png'), 'Resolution', 300);
447         exportgraphics(axFFT_Ib, fullfile(folder, '9_FFT_Current_PhaseB.png'), 'Resolution', 300);
448         exportgraphics(axFFT_Ic, fullfile(folder, '10_FFT_Current_PhaseC.png'), 'Resolution', 300);
449
450         % Tab 4: Power
451         exportgraphics(axPower, fullfile(folder, '11_Power_Flow.png'), 'Resolution', 300);
452
453         % Tab 5: Gates
454         exportgraphics(axG1, fullfile(folder, '12_Gate_T1.png'), 'Resolution', 300);
455         exportgraphics(axG2, fullfile(folder, '13_Gate_T2.png'), 'Resolution', 300);
456         exportgraphics(axG3, fullfile(folder, '14_Gate_T3.png'), 'Resolution', 300);
457         exportgraphics(axG4, fullfile(folder, '15_Gate_T4.png'), 'Resolution', 300);
458
459         % Tab 6: Trajectory
460         exportgraphics(axTraj, fullfile(folder, '16_Trajectory_Plot.png'), 'Resolution', 300);
461

```

```

462     uialert(fig, 'All Plots and Report Saved Successfully!', 'Export Done');
463     catch ME
464         uialert(fig, ME.message, 'Export Error');
465     end
466 end
467
468 function exportData(~, ~)
469     assignin('base', 'sim_time', sim_t);
470     assignin('base', 'sim_V_pole', sim_Vao);
471     assignin('base', 'sim_V_phase', sim_Van);
472     assignin('base', 'sim_V_line', sim_Vab);
473     assignin('base', 'sim_Currents_3Ph', sim_Ia);
474     assignin('base', 'sim_Gate_Signals', sim_Gates);
475     assignin('base', 'sim_Inst_Power', sim_Power);
476     uialert(fig, 'Variables exported to MATLAB Workspace.', 'Export Complete');
477 end
478
479 function F = she_equations(alpha, Vdc, Vref)
480     a = alpha;
481     F = [(2*Vdc/pi)*(cos(a(1)) - cos(a(2)) + cos(a(3))) - Vref;
482         cos(5*a(1)) - cos(5*a(2)) + cos(5*a(3));
483         cos(7*a(1)) - cos(7*a(2)) + cos(7*a(3))];
484 end
485
486 function v = generate_pwm(t, alpha, Vdc, ph, Freq)
487     v = zeros(size(t));
488     w_t = mod(2*pi*Freq*t + ph, 2*pi);
489     a = alpha;
490
491     mask_pos = (w_t < pi);
492     theta = w_t;
493
494     v(mask_pos & (theta < pi/2) & ((theta >= a(1) & theta < a(2)) | theta >= a(3))) =
Vdc/2;
495     tm = pi - theta;
496     v(mask_pos & (theta >= pi/2) & ((tm >= a(1) & tm < a(2)) | tm >= a(3))) = Vdc/2;
497
498     mask_neg = ~mask_pos;
499     tn = w_t - pi;
500     v(mask_neg & (tn < pi/2) & ((tn >= a(1) & tn < a(2)) | tn >= a(3))) = -Vdc/2;
501     tnm = pi - tn;
502     v(mask_neg & (tn >= pi/2) & ((tnm >= a(1) & tnm < a(2)) | tnm >= a(3))) = -Vdc/2;
503 end
504
505 function [freqs, mag] = get_fft(signal, Ts, F_fund)
506     L = length(signal); Y = fft(signal); P2 = abs(Y/L);
507     P1 = P2(1:floor(L/2)+1); P1(2:end-1) = 2*P1(2:end-1);
508     harm_orders = 1:25; mag = zeros(size(harm_orders)); freqs = harm_orders * F_fund;
509     freq_axis = (0:(L/2))/L * (1/Ts);
510     for k = 1:length(harm_orders)
511         [~, idx] = min(abs(freq_axis - freqs(k))); mag(k) = P1(idx);
512     end
513 end
514 end

```

Listing 2: SHE_Inverter_Task_Final.m